

Into the World of MLSecOps or Machine Learning Security Operations

Mohammad Fozouni (Ph.D.)

MLSECOPS IN ACTION COURSE

5 ستون از امال سک آپس



5 ستون از امال سک آپس



5 ستون از امال سک آپس



-  Supply Chain Vulnerability
-  Model Provenance
-  Governance, Risk & Compliance
-  Trusted AI
-  Adversarial Machine Learning

لایه

زیرساخت و وابستگی‌ها

منشأ و زنجیره اعتماد

قوانین و سیاست‌ها

کیفیت و عدالت مدل

مقاومت در برابر حمله مستقیم

5 ستون از امال سک آپس

لایه

زیرساخت و وابستگی‌ها

منشأ و زنجیره اعتماد

قوانین و سیاست‌ها

کیفیت و عدالت مدل

مقاومت در برابر حمله مستقیم

1. تمام اجزای خارجی که در ساخت مدل استفاده می‌شن؛ کتابخانه‌ها، دیتاست‌ها، مدل‌های پیش‌آموزش‌دیده و ایمپچ‌های داکر.

2. ردیابی اینکه مدل از کجا اومده، کی ساخته، با چه داده‌ای آموزش دیده، و چه تغییراتی روش انجام شده.

3. قوانین، سیاست‌ها و استانداردهایی که مشخص می‌کنن مدل چطور باید ساخته و استفاده بشه.

4. مدل باید قابل اعتماد باشه؛ عادلانه، شفاف، قابل توضیح، و بدون سوگیری عمل کنه.

5. محافظت از مدل در برابر ورودی‌های فریبنده که مهاجم می‌سازه تا مدل رو گول بزنه.



Supply Chain Vulnerability



Model Provenance



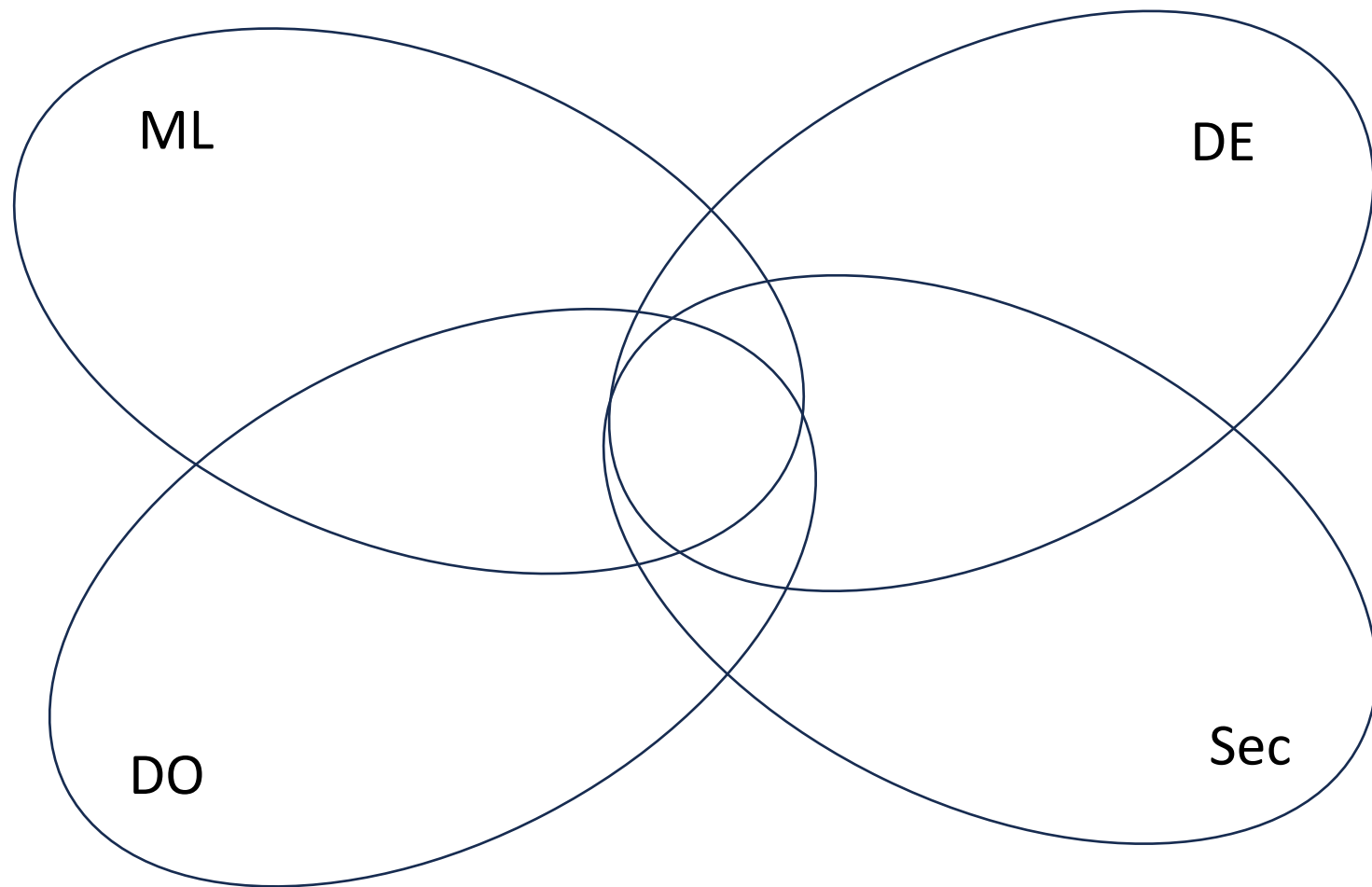
Governance, Risk & Compliance

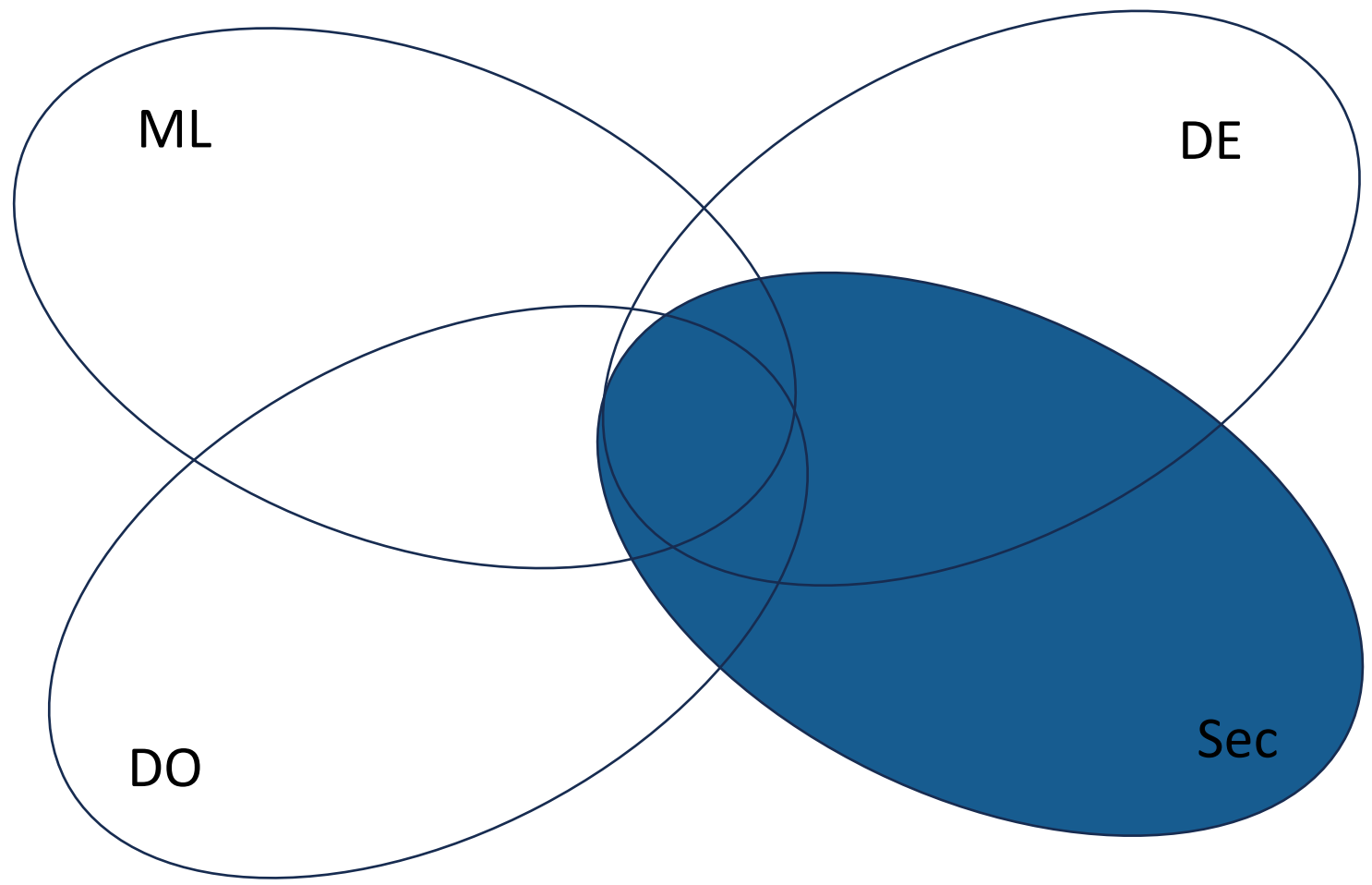


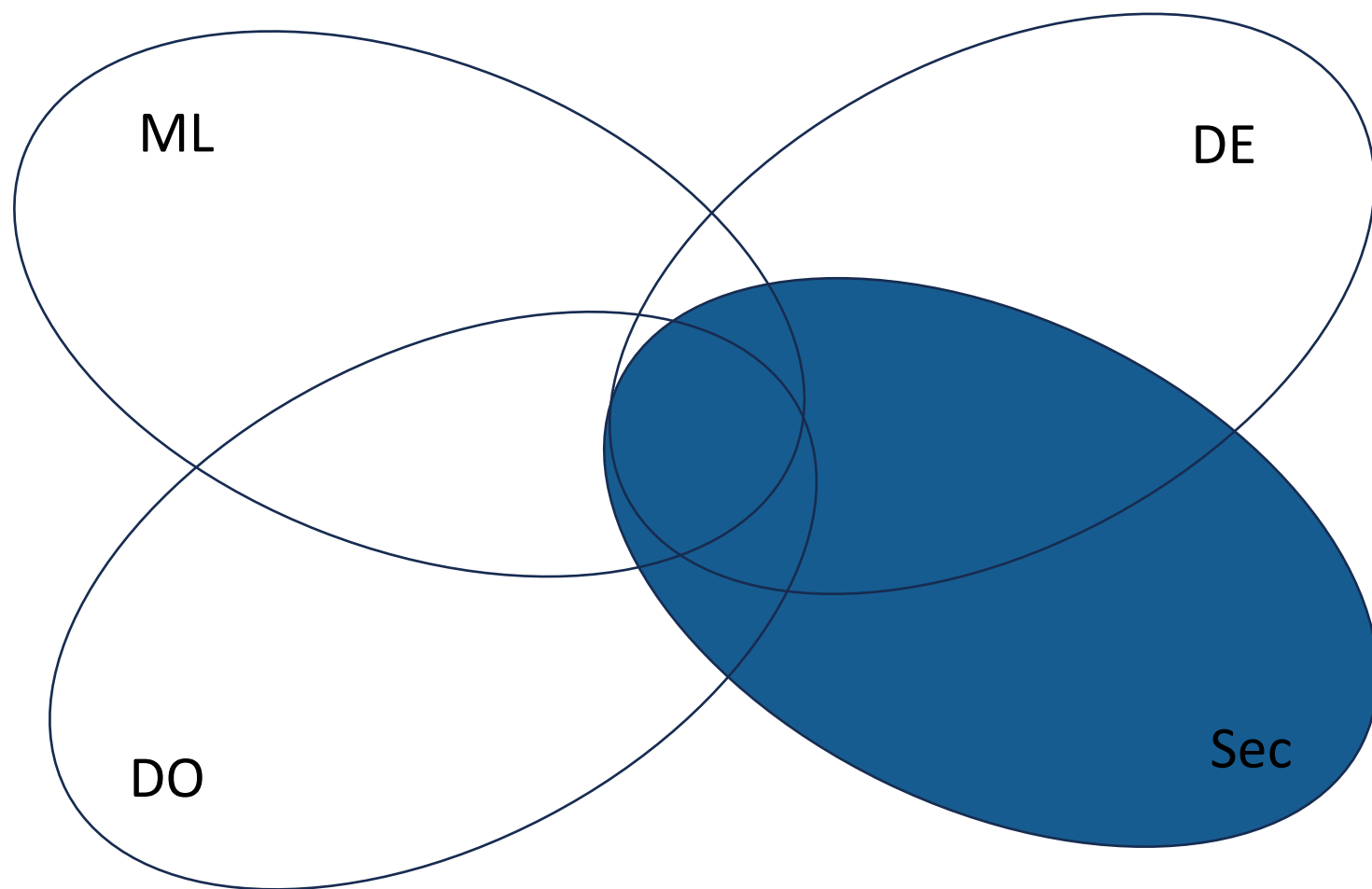
Trusted AI



Adversarial Machine Learning







امنیت در این حوزه، به هیچ وجه به امنیت سایبری به معنی سنتی خود اشاره نمی‌کند. بلکه منظور امنیت مدل‌های یادگیری ماشین است که وارد محیط پروداکشن و عملیاتی سازمان شده‌اند.



1ST EDITION

Adversarial AI Attacks, Mitigations, and Defense Strategies

A cybersecurity professional's guide to AI attacks, threat modeling, and securing AI with MLSecOps

JOHN SOTIROPOULOS

· Table of Contents

· Preface

· Part 1: Introduction to Adversarial AI

· Chapter 1: Getting Started with AI

· Chapter 2: Building Our Adversarial Playground

· Chapter 3: Security and Adversarial AI

· Part 2: Model Development Attacks

· Chapter 4: Poisoning Attacks

· Chapter 5: Model Tampering with Trojan Horses and Model Reprogramming

· Chapter 6: Supply Chain Attacks and Adversarial AI

· Part 3: Attacks on Deployed AI

· Chapter 7: Evasion Attacks against Deployed AI

· Chapter 8: Privacy Attacks – Stealing Models

· Chapter 9: Privacy Attacks – Stealing Data

· Chapter 10: Privacy-Preserving AI

· Part 4: Generative AI and Adversarial Attacks

· Chapter 11: Generative AI – A New Frontier

· Chapter 12: Weaponizing GANs for Deepfakes and Adversarial Attacks

· Chapter 13: LLM Foundations for Adversarial AI

· Chapter 14: Adversarial Attacks with Prompts

· Chapter 15: Poisoning Attacks and LLMs

· Chapter 16: Advanced Generative AI Scenarios

· Part 5: Secure-by-Design AI and MLSecOps

· Chapter 17: Secure by Design and Trustworthy AI

· Chapter 18: AI Security with MLSecOps

· Chapter 19: Maturing AI Security

· Index



1ST EDITION

Adversarial AI Attacks, Mitigations, and Defense Strategies

A cybersecurity professional's guide to AI attacks, threat modeling, and securing AI with MLSecOps

JOHN SOTIROPOULOS

· Table of Contents

· Preface

· Part 1: Introduction to Adversarial AI

· Chapter 1: Getting Started with AI

· Chapter 2: Building Our Adversarial Playground

· Chapter 3: Security and Adversarial AI

· Part 2: Model Development Attacks

· Chapter 4: Poisoning Attacks

· Chapter 5: Model Tampering with Trojan Horses and Model Reprogramming

· Chapter 6: Supply Chain Attacks and Adversarial AI

· Part 3: Attacks on Deployed AI

· Chapter 7: Evasion Attacks against Deployed AI

· Chapter 8: Privacy Attacks – Stealing Models

· Chapter 9: Privacy Attacks – Stealing Data

· Chapter 10: Privacy-Preserving AI

· Part 4: Generative AI and Adversarial Attacks

· Chapter 11: Generative AI – A New Frontier

· Chapter 12: Weaponizing GANs for Deepfakes and Adversarial Attacks

· Chapter 13: LLM Foundations for Adversarial AI

· Chapter 14: Adversarial Attacks with Prompts

· Chapter 15: Poisoning Attacks and LLMs

· Chapter 16: Advanced Generative AI Scenarios

· Part 5: Secure-by-Design AI and MLSecOps

· Chapter 17: Secure by Design and Trustworthy AI

· Chapter 18: AI Security with MLSecOps

· Chapter 19: Maturing AI Security

· Index

بهترین کتابخانه‌ی پایتون در دنیای MLSecOps

بهترین کتابخانه‌ی پایتون در دنیای MLSecOps



Adversarial
Robustness
Toolbox

An open-source Python framework for evaluating and defending machine-learning systems

ART Is More Than an Attack Library. It provides the following main phases for us

Attack >>>> Defense >>>> Evaluation



Why ART matters in MLSecOps:

1. Provides a common framework for **attacking, evaluating, and defending ML models**
2. Supports multiple ML frameworks and model types
3. Allows security teams to simulate realistic **adversarial ML threats**
4. Covers the ML security lifecycle:
Attack >>>> Measure >>>> Defend >>>> Re-test
5. Turns adversarial ML research techniques into **practical security testing**

Key idea:

ART lets us treat an ML model like a security target. Indeed, attack it, measure its weaknesses, apply defenses, and test again.

ART covers a broad range of adversarial ML attacks

Attack Category	What ART Can Test
Evasion	Modify an input to fool a trained model

مهاجم ورودی را کمی تغییر می‌دهد تا مدل تصمیم اشتباه بگیرد، بدون اینکه تغییر برای انسان خیلی قابل تشخیص باشد.



Adversarial
Robustness
Toolbox

ART covers a broad range of adversarial ML attacks

Attack Category	What ART Can Test
Evasion	Modify an input to fool a trained model
Poisoning	Manipulate training data to compromise learning

مهاجم داده‌های آموزشی را خراب یا دستکاری می‌کند تا مدل از ابتدا چیز اشتباهی یاد بگیرد.



Adversarial
Robustness
Toolbox

ART covers a broad range of adversarial ML attacks

Attack Category	What ART Can Test
Evasion	Modify an input to fool a trained model
Poisoning	Manipulate training data to compromise learning
Extraction	Attempt to steal information about a model

مهاجم با فرستادن تعداد زیادی درخواست به مدل سعی می‌کند رفتار یا دانش مدل را یاد بگیرد و حتی یک مدل مشابه بسازد.



Adversarial
Robustness
Toolbox

ART covers a broad range of adversarial ML attacks

Attack Category	What ART Can Test
Evasion	Modify an input to fool a trained model
Poisoning	Manipulate training data to compromise learning
Extraction	Attempt to steal information about a model
Inference	Investigate information leakage from ML models

مهاجم از خروجی‌های مدل، اطلاعاتی درباره‌ی داده‌های آموزشی یا اطلاعات حساس موجود در آن‌ها استخراج می‌کند.



Robustness
Toolbox

ART covers a broad range of adversarial ML attacks

Attack Category	What ART Can Test
Evasion	Modify an input to fool a trained model
Poisoning	Manipulate training data to compromise learning
Backdoors	Test models containing hidden malicious behavior

مدل در ظاهرا درست کار می‌کند، اما یک شرط مخفی (Trigger) داخل آن وجود دارد که وقتی فعال شود، مدل را به سمت یک رفتار خاص و مهاجم‌پسند می‌برد.



Adversarial
Robustness
Toolbox

ورودی را تغییر می‌دهیم تا مدل را گول بزنیم

Evasion

داده آموزشی را خراب می‌کنیم تا مدل اشتباه یاد می‌گیرد

Poisoning

زیاد از مدل سؤال می‌کنیم تا رفتار یا دانش مدل را تقلید کنیم

Extraction

خروجی مدل را تحلیل می‌کنیم تا اطلاعات مخفی مدل را حدس بزنیم

Inference

تریگر مخفی وارد مدل می‌کنیم تا با تحریک آن رفتار خاص در مدل ایجاد شود

Backdoor



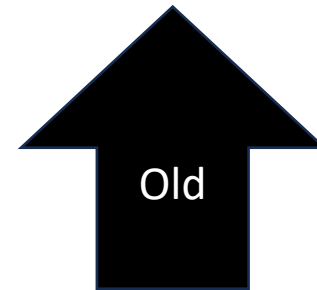
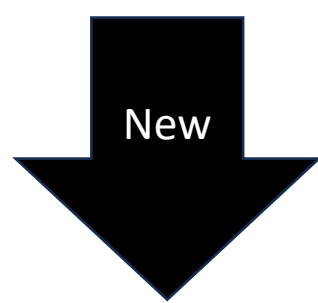
MLSecOps

In MLSecOps, adversarial attacks can be broadly considered across two realms:

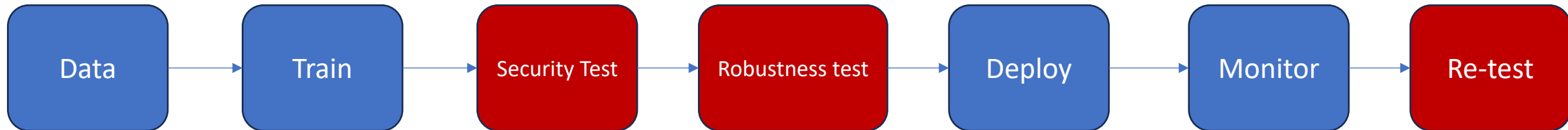
1. Traditional/predictive ML models
2. Generative AI/LLMs.

The five attack categories we have mentioned (Evasion, Poisoning, Extraction, Inference, and Backdoor) can occur in both realms, although their specific techniques and implementations differ.

Traditional MLOps



Modern MLOps with MLSecOps practices



ART enables MLSecOps teams to ask

1. Can an attacker fool my model?
2. How much perturbation is required?
3. Which attacks are most effective?
4. Does my defense actually improve robustness?
5. Does a model remain secure after retraining or modification?



Chapter 3: **Security and** **Adversarial AI**



Security Fundamentals: CIA triad

سه‌گانه CIA یک مدل امنیتی است که سه اصل اساسی امنیت اطلاعات را توصیف می‌کند:

محرمانگی (Confidentiality): اطلاعات فقط برای افراد مجاز قابل دسترسی باشد (مثل رمزنگاری و کنترل دسترسی).

یکپارچگی (Integrity): اطلاعات دقیق و دست‌نخورده باقی بماند و فقط افراد مجاز بتوانند آن را تغییر دهند (مثل امضای دیجیتال و هش).

در دسترس بودن (Availability): اطلاعات و سیستم‌ها هر زمان که افراد مجاز نیاز دارند، قابل دسترسی باشند (مثل پشتیبان‌گیری و محافظت در برابر حملات DDoS؛ حملات DDoS سرویس را از دسترس خارج می‌کنند).

این سه اصل پایه‌ی طراحی هر سیستم امنیتی هستند و نقض هر کدام به معنای آسیب‌پذیری سیستم است



Threat Modeling

روشی ساختاریافته که به منظور شناخت، اولویت‌بندی و مدیریت مخاطرات در یک سیستم بکار گرفته می‌شود. روش‌هایی برای تعیین مخاطرات وجود دارند. اما 2 روش ذیل متمایز (و محبوب) عمل می‌کنند:

1- STRIDE

چه چیزی ممکن است خراب شود؟

2- Attack Tree

مهاجم چگونه وارد می‌شود؟

STRIDE methodology

S Spoofing

Can an attacker gain access using a false identity?

جعل هویت: وانمود کردن به هویت فرد یا سیستم دیگر

T Tampering

Can an attacker modify data as it flows through the application?

دستکاری: تغییر غیرمجاز داده‌ها یا کد

R Repudiation

If an attacker denies doing something, can we prove he did it?

انکار: انکار انجام یک عمل بدون امکان اثبات خلاف آن

I Information disclosure

Can an attacker gain access to private or potentially injurious data?

افشای اطلاعات: دسترسی غیرمجاز به داده‌های محرمانه

D Denial of service

Can an attacker crash or reduce the availability of the system?

انکار سرویس: از دسترس خارج کردن سرویس برای کاربران مجاز

E Elevation of privilege

Can an attacker assume the identity of a privileged user?

ارتقای سطح دسترسی: کسب دسترسی‌های بیشتر از حد مجاز

STRIDE methodology

Authentication

Integrity

Non-repudiation

Confidentiality

Availability

Authorization

S

Spoofing

Can an attacker gain access using a false identity?

T

Tampering

Can an attacker modify data as it flows through the application?

R

Repudiation

If an attacker denies doing something, can we prove he did it?

I

Information disclosure

Can an attacker gain access to private or potentially injurious data?

D

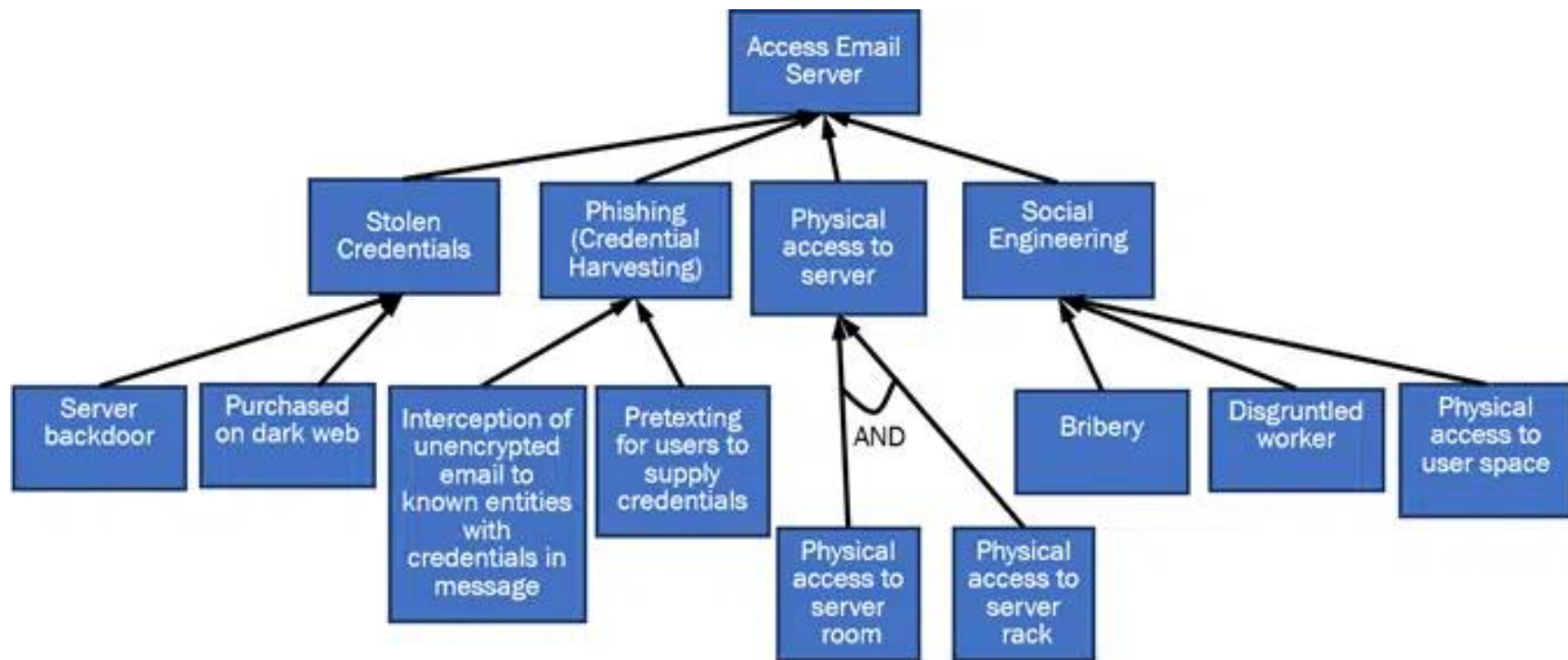
Denial of service

Can an attacker crash or reduce the availability of the system?

E

Elevation of privilege

Can an attacker assume the identity of a privileged user?



درخت حمله:

درخت حمله یک نمودار درختی است که نشان می‌دهد یک مهاجم برای رسیدن به هدفش، از چه مسیرها و مراحل می‌تواند استفاده کند.

- ریشه درخت = هدف نهایی مهاجم (مثل دسترسی به حساب بانکی کاربر).
- شاخه‌ها = راه‌های مختلف برای رسیدن به آن هدف.
- برگ‌ها = اقدام‌های مشخص و کوچکی که مهاجم باید انجام دهد.

حالا که خطرات را شناختیم ...

باید به هر کدومشون یک ریسکی رو تخصیص بدهیم. بطور کلی این ریسک را بسته به میزان احتمال رخداد خطر و تاثیری که روی سیستم ما دارد، تخمین خواهیم زد.

وقتی که از یک خطر یا تهدید صحبت می‌کنیم، به واژه‌ای با عنوان Mitigation (کاهش) نیز برخورد خواهیم کرد؛ یعنی اینکه ما چگونه باید در مقابل این تهدیدات دفاع نموده و آسیب‌پذیری را مرتفع کنیم.

هر اقدامی که ریسک را کم کند = Mitigation

۱. قبل از وقوع حمله (پیشگیری):

هنوز خطری اتفاق نیفتاده، ولی می‌دانیم ممکن است بیفتد، پس جلوییش را می‌گیریم.
مثال: نصب فایروال، فعال کردن MFA، رمزنگاری داده

۲. بعد از شناسایی ضعف (رفع):

خطر هنوز اتفاق نیفتاده، ولی آسیب‌پذیری پیدا کرده‌ایم و می‌بندیمش.
مثال: وصله کردن (پچ) یک باگ، بستن یک پورت باز

۳. بعد از وقوع حمله (کاهش اثر و جمع کردن):

حمله اتفاق افتاده، حالا می‌خواهیم خسارت را کم کنیم و سیستم را برگردانیم به روز قبل از حادثه.
مثال: بازیابی از پشتیبان، قطع دسترسی مهاجم، اطلاع‌رسانی به کاربران

How we can secure our stack?

Host Security

Regular updates	<code>sudo apt update ; sudo apt upgrade</code>
Minimal software	هر چقدر نرم افزارهای کمتری داشته باشیم، احتمال وجود باگ امنیتی هم کمتر خواهد شد
User access control	دسترسی کاربران را تا حد امکان محدود کنید
Firewall configuration	استفاده از iptables و باز کردن پورت‌هایی که واقعن مورد نیاز هستند
Container security	استفاده از فلگ‌هایی مانند <code>--cap-drop</code> برای محدود کردن قابلیت‌های کانتینر. استفاده از SBOM برای بررسی آرتیفکت‌ها

Host Security

System monitoring and auditing	استفاده از ابزارهایی مانند Logwatch یا Fail2ban
Backup and recovery	بطور مرتب از دیتاهای حساس بک‌آپ بگیرید و در جایی خارج از دیتاستر اصلی نگهداری کنیم
Disable unused network services	ابتدا <code>netstat -tuln</code> و سپس هر سرویس که لازم نیست را غیرفعال کنید
SSH access	بجای رمز عبور از کلید SSH استفاده کنید. پورت پیش فرض SSH را عوض کنید
Endpoint security	استفاده از ClamAV برای مقابله با بدافزارها و Lynis برای هاردن کردن هاست
Vulnerability management	بطور مرتب و متناوب به بررسی و شناسایی و کاهش آسیب‌پذیریهای نرم‌افزارهای موجود در هاست بپردازیم

Network Protection

1. فقط سرویس‌های ضروری در معرض (expose) قرار بگیرند (بشوند).
 2. HTTPS برای جلوگیری از استراق سمع (Eavesdropping)، جعل هویت (Spoofing) و مرد میانی (MITM) باید اجباری باشد.
 3. استفاده از NGINX به عنوان پروکسی برای:
 - ریدایرکت HTTP به HTTPS
 - فقط اجازه‌ی دسترسی به endpoint های وضعیت یا status بدهیم
 - مسدود کردن دسترسی مستقیم به API
 - Rate Limiting برای مقابله با DoS
 4. فایروال روی میزبان: مسدود کردن پورت‌های 8000 و 5000 از بیرون.
- به کمک nginx سرویس‌های غیر ضرور و پورت‌های نا لازم رو مدیریت می‌کنیم.**

Authentication

1. OAuth2 با GitHub برای ورود کاربران به web app.
2. API Key بین service_app و service_api (فقط service_app می‌تواند به API درخواست بدهد).

برای دسترسی به وب‌اپ خودمون باید یک سیستم Auth ایجاد کنیم که همگان راحت بهش دسترسی بگیرند.

Data Protection

1. داده در حال انتقال با TLS محافظت می‌شود.
2. داده در حالت سکون (model) با AES-256-GCM رمزنگاری می‌شود.
3. کلید با OpenSSL تولید و در پوشه keys ذخیره می‌شود (در پروداکشن از Vault یا Secrets Manager استفاده شود).

به منظور بررسی Integrity از روی فایل رمز شده‌ی مدل یک هش می‌سازیم. حتمن در محیط پروداکشن پیش از رمزنگاری، فایل را با هش مذکور مقایسه می‌کنیم و اگر تغییری در آن ایجاد نشده بود، مدل را live می‌کنیم.

Access Control

1. اصل کمترین امتیاز (Least Privilege) و RBAC را رعایت می‌کنیم.
 2. ایجاد یک گروه (مثلن) ml-ops و ساخت کاربران در این گروه با دسترسی محدود.
 3. تنظیم مجوزهای Linux مثل 750 و 770 برای keys، ssl و مدل‌ها.
 4. اجرای کانتینرها با کاربران اختصاصی نه کاربر root.
- کاربرانی می‌سازیم که دسترسی‌های خاصی دارند. این کار رو بطور کامل با کامندهای لینوکس می‌زنیم.
 - هدف اینه که پوشه‌های مهم رو فقط به کاربران خاصی که واقعن نیاز دارند بدهیم. نه کسی دیگه.
 - **هیچ وقت دسترسی زیاد نگیریم. مسئولیت رفتن تو گونی رو باید قبول کنیم.**

Code and Artifacts

1. SAST با Bandit: اسکن کد پایتون برای آسیب‌پذیری‌ها.
2. SCA با Trivy: اسکن وابستگی‌ها و ایمیج‌های داکر.
 - آسیب‌پذیری در Pillow 9.2.0 با ارتقا رفع شد.
 - آسیب‌پذیری بحرانی در openssh-client با حذف پکیج یا استفاده از ایمیج slim رفع شد.
3. Secret Scanning با Trivy: کشف کلید خصوصی در سورس‌کد؛ رفع با gitignore.

SAST = Static Application Security Testing

Analyzes source code *without running it* to find vulnerabilities (Bandit for Python).

SCA = Software Composition Analysis

Scans third-party libraries, dependencies, and container images for known vulnerabilities (Trivy).

Securing Jupyter Notebooks

1. Notebook ها هم کد و هم وابستگی دارند و در نتیجه باید اسکن شوند.

2. ابزارهای موجود در این راستا:

- Bandit پس از تبدیل به .py با nbconvert.
- NBDefense برای تشخیص کد مخرب و PII.

فایل های ipynb. خودمون رو هم باید اسکن کنیم که بعدن حسرتش رو نخوریم.

Securing Models from Malicious Code

1. فرمت Pickle خطرناک است: می‌تواند هنگام بارگذاری کد دلخواه اجرا کند.
2. جایگزین امن‌تر پیدا کنید (بدلیل بروزرسانی‌های مداوم نام نمی‌برم).
3. ModelScan برای اسکن مدل‌های سریالایز شده (H5، Pickle) و کشف کد مخرب عالیست.

```
```python
pip install modelscan
modelscan -p models
```
```

Adversarial AI

1. کنترل‌های امنیتی سنتی از محیط و مصنوعات محافظت می‌کنند، اما منطق درونی مدل (brain) را محافظت نمی‌کنند.
2. بعنوان مثال، در حمله‌ی Evasion هکر با افزودن نویز نامرئی (perturbation) به تصویر، باعث می‌شود که مدل آن تصویر را اشتباه طبقه‌بندی کند.
3. نمونه‌های واقعی: نوار چسب روی تابلو سرعت برای فریب خودروی هوشمند، عینک برای فریب تشخیص چهره.



Traditional security vs adversarial AI

| امنیت سنتی | هوش مصنوعی خصمانه |
|---|--|
| واکنشی (Reactive) (باید اتفاق بیوفته و بعد بریم دنبال چاره) | پیشگیرانه و تحلیلی (بطور کلی قبل از اتفاق افتادن دنبال جلوگیری از رخ دادنش هستیم) |
| مبتنی بر امضا (Signature) (الگوهای ثابت از حمله‌های قبلی دنبال می‌شه) | مبتنی بر بهینه‌سازی ورودی (مهاجم دنبال تغییر در کوچکترین مورد ممکن برای گول زدن مدل هست) |
| داده/شبکه/محافظت از سیستم | محافظت از منطق مدل |
| در برابر حملات شناخته‌شده | در برابر حملات نو و تطبیقی |

Adversarial Robustness مکمل ضروری امنیت سنتی است و باید به تیم دفاعی سازمان یا مجموعه، حتمن اضافه گردد. یکی محافظ در ورودی است و دیگری محافظ مغز.



MLSecOps

See you all in the next session 🕶️